

✓ 14 — Multi-session layer/Cre-line geometry statistics

This notebook reconstructs embedding geometry across all available sessions, merges session metadata, runs bootstrapped uncertainty summaries, and gates layer/Cre-line claims.

✓ Setup and asset index

```
from pathlib import Path
import os
import sys
import yaml
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Edit this path only if your project has moved.
PROJECT_ROOT = Path(
    os.environ.get(
        "LATENT_MANIFOLD_PROJECT_ROOT",
        r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural"
    )
).resolve()

SRC_DIR = PROJECT_ROOT / "src"
if not PROJECT_ROOT.exists():
    raise FileNotFoundError(f"PROJECT_ROOT does not exist: {PROJECT_ROOT}")
if not SRC_DIR.exists():
    raise FileNotFoundError(f"src directory does not exist: {SRC_DIR}")
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
os.chdir(PROJECT_ROOT)

cfg_path = PROJECT_ROOT / "configs" / "publication_upgrade.yaml"
if not cfg_path.exists():
    raise FileNotFoundError(f"Could not find publication config: {cfg_path}")
with cfg_path.open("r", encoding="utf-8") as f:
    pub_cfg = yaml.safe_load(f)
    cfg = pub_cfg

# Versioned namespace. This prevents accidental overwriting of older publicati
PUBLICATION_RUN_LABEL = os.environ.get("PUBLICATION_RUN_LABEL", "publication_u
ALLOW_CANONICAL_PUBLICATION_WRITE = os.environ.get("ALLOW_CANONICAL_PUBLICATION")
```

```

if PUBLICATION_RUN_LABEL == "publication_upgrade" and not ALLOW_CANONICAL_PUBLICATION_UPGRADE:
    raise RuntimeError(
        "Refusing to write into the canonical publication_upgrade namespace. "
        "Use a versioned PUBLICATION_RUN_LABEL or set ALLOW_CANONICAL_PUBLICATION_UPGRADE"
    )

class ProjectPaths:
    def __init__(self, root: Path, run_label: str):
        self.root = root
        self.configs_dir = root / "configs"
        self.data_dir = root / "data"
        self.external_dir = root / "data" / "external"
        self.raw_dir = root / "data" / "raw"
        self.interim_dir = root / "data" / "interim"
        self.processed_dir = root / "data" / "processed"
        self.versioned_processed_dir = self.processed_dir / run_label
        self.reports_dir = root / "reports"
        self.tables_dir = root / "reports" / "tables"
        self.figures_dir = root / "reports" / "figures"
        self.html_dir = root / "reports" / "html"
        self.publication_tables_dir = root / "reports" / "tables" / run_label
        self.publication_figures_dir = root / "reports" / "figures" / run_label
        self.manuscript_dir = root / "manuscript" / "top_journal_scaffold"
        for d in [
            self.versioned_processed_dir,
            self.publication_tables_dir,
            self.publication_figures_dir,
            self.manuscript_dir,
        ]:
            d.mkdir(parents=True, exist_ok=True)

paths = ProjectPaths(PROJECT_ROOT, PUBLICATION_RUN_LABEL)

def save_table(df, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)
    return path

def save_figure(fig, path, dpi=300):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fig.savefig(path, dpi=dpi, bbox_inches="tight")
    return path

# Upgrade helper functions shipped with this patch.
from v1_manifold_publication.multisession_core import (
    build_multisession_asset_index,
    candidate_feature_files,
    choose_best_feature_table,
    ready_sessions,

```

```

        movie_feature_targets,
        empirical_null_summary,
        safe_read_csv,
        safe_table_index,
        claim_gate_from_assets,
        ensure_table,
    )

    metadata_path = paths.external_dir / "allen_v1_natural_movie_experiments.csv"
    asset_index = build_multisession_asset_index(paths, metadata_path=metadata_path)
    save_table(asset_index, paths.publication_tables_dir / "00_multisession_asset_index")

    print("Using PROJECT_ROOT:", PROJECT_ROOT)
    print("Publication run label:", PUBLICATION_RUN_LABEL)
    print("Publication tables:", paths.publication_tables_dir)
    print("Publication figures:", paths.publication_figures_dir)
    print("Versioned processed dir:", paths.versioned_processed_dir)
    display(asset_index)

```

```

Using PROJECT_ROOT: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication run label: publication_upgrade_v3_multisession
Publication tables: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication figures: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Versioned processed dir: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold

```

	session_id	feature_file	feature_shape	
0	500855614	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\500855614.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\500855614.csv
1	500964514	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\500964514.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\500964514.csv
2	501271265	C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\501271265.csv	(900, 105)	\Users\Peter\Documents\projects\NeuroAI\latent-manifold\assets\501271265.csv

Reconstruct geometry summaries from all saved embeddings

```

# -----
# Notebook 14: reconstruct geometry time series for common embeddings
..

```

```

# -----

from v1_manifold_publication.geometry_publication import (
    geometry_timeseries,
    summarize_geometry_ts,
)

common_embeddings = ["pca", "pca_full", "umap", "isomap"]

rows = []
ts_status = []

ready = (
    asset_index
    .loc[asset_index["has_embeddings"], "session_id"]
    .astype(str)
    .tolist()
)

print("Sessions with embeddings:", ready)
print("Common embeddings used for multi-session geometry:", common_embedding

for session_id in ready:
    emb_path = Path(
        asset_index.loc[
            asset_index["session_id"].astype(str) == str(session_id),
            "embedding_file",
        ].iloc[0]
    )

    print("\n" + "=" * 90)
    print("Session:", session_id)
    print("Embedding file:", emb_path)

    try:
        with np.load(emb_path, allow_pickle=False) as emb:
            available = [
                name for name in common_embeddings
                if name in emb.files
            ]

            print("Available common embeddings:", available)

            for name in available:
                Z = emb[name]

                if Z.ndim != 2 or Z.shape[0] <= 10 or Z.shape[1] < 2:
                    ts_status.append({
                        "session_id": session_id,
                        "embedding": name,
                        "status": "skipped",

```

```

        "reason": f"Invalid embedding shape: {Z.shape}",
    })
    continue

    geom_ts = geometry_timeseries(Z)
    geom_ts = geom_ts.copy()
    geom_ts["session_id"] = session_id
    geom_ts["embedding"] = name

    save_table(
        geom_ts,
        paths.publication_tables_dir
        / f"14_geometry_timeseries_session_{session_id}_{name}.c
    )

    summary = summarize_geometry_ts(geom_ts)

    pivot = (
        summary
        .set_index("metric")[["median", "p99", "max"]]
        .stack()
        .to_dict()
    )

    rows.append({
        "session_id": session_id,
        "embedding": name,
        "n_frames": int(Z.shape[0]),
        "embedding_dim": int(Z.shape[1]),
        **{f"{k[0]}_{k[1]}": v for k, v in pivot.items()},
    })

    ts_status.append({
        "session_id": session_id,
        "embedding": name,
        "status": "completed",
        "n_frames": int(Z.shape[0]),
        "embedding_dim": int(Z.shape[1]),
    })

except Exception as exc:
    ts_status.append({
        "session_id": session_id,
        "embedding": None,
        "status": "failed",
        "reason": repr(exc),
    })

geom = pd.DataFrame(rows)
geometry_status = pd.DataFrame(ts_status)

```

```
save_table(
    geom,
    paths.publication_tables_dir / "publication_reconstructed_embedding_geom"
)

save_table(
    geom,
    paths.publication_tables_dir / "14_geometry_with_session_metadata.csv",
)

save_table(
    geometry_status,
    paths.publication_tables_dir / "14_geometry_reconstruction_status.csv",
)

display(geom.head(20))
display(geometry_status)
```

Sessions with embeddings: ['500855614', '500964514', '501271265']
Common embeddings used for multi-session geometry: ['pca', 'pca_full', 'umap']

=====
Session: 500855614
Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']

=====
Session: 500964514
Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']

=====
Session: 501271265
Embedding file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-
Available common embeddings: ['pca', 'pca_full', 'umap', 'isomap']

	session_id	embedding	n_frames	embedding_dim	speed_median	speed_p99
0	500855614	pca	900	3	1.005158	2.503851
1	500855614	pca_full	900	20	2.677863	4.246316
2	500855614	umap	900	3	0.082220	0.752321
3	500855614	isomap	900	3	1.200051	5.479009
4	500964514	pca	900	3	1.121127	3.401901
5	500964514	pca_full	900	20	3.003222	4.910268
6	500964514	umap	900	3	0.080179	0.437021
7	500964514	isomap	900	3	3.832981	12.892728
8	501271265	pca	900	3	1.186652	3.226026

9	501271265	pca_full	900	20	2.878449	4.561172
10	501271265	umap	900	3	0.076037	0.562987
11	501271265	isomap	900	3	3.932050	12.270895
	session_id	embedding	status	n_frames	embedding_dim	
0	500855614	pca	completed	900	3	
1	500855614	pca_full	completed	900	20	
2	500855614	umap	completed	900	3	
3	500855614	isomap	completed	900	3	
4	500964514	pca	completed	900	3	
5	500964514	pca_full	completed	900	20	
6	500964514	umap	completed	900	3	
7	500964514	isomap	completed	900	3	
8	501271265	pca	completed	900	3	
9	501271265	pca_full	completed	900	20	
10	501271265	umap	completed	900	3	
11	501271265	isomap	completed	900	3	

```
# -----
# Notebook 14: multi-session geometry summary with bootstrap CIs
# -----

from v1_manifold_publication.stats import bootstrap_ci

geom_path = paths.publication_tables_dir / "14_geometry_with_session_metadata"

if not geom_path.exists():
    raise FileNotFoundError(f"Missing geometry table: {geom_path}")

geom = pd.read_csv(geom_path)

common_embeddings = ["pca", "pca_full", "umap", "isomap"]

geom = geom[geom["embedding"].isin(common_embeddings)].copy()

value_cols = [
    c for c in geom.columns
    if c.endswith("_median") or c.endswith("_p99") or c.endswith("_max")
]
```

```

summary_rows = []

for embedding, sub in geom.groupby("embedding", dropna=False):
    for col in value_cols:
        values = pd.to_numeric(sub[col], errors="coerce").dropna().to_numpy()

        if len(values) == 0:
            continue

        ci = bootstrap_ci(
            values,
            n_bootstrap=int(cfg.get("models", {}).get("n_bootstrap", 1000)),
            rng=cfg["project"]["random_seed"],
        )

        summary_rows.append({
            "embedding": embedding,
            "metric": col,
            "n_sessions": int(sub["session_id"].nunique()),
            "mean": float(np.mean(values)),
            "median": float(np.median(values)),
            "std": float(np.std(values, ddof=1)) if len(values) > 1 else 0.0,
            "ci_low": ci["ci_low"],
            "ci_high": ci["ci_high"],
        })

geometry_multisession_summary = pd.DataFrame(summary_rows)

save_table(
    geometry_multisession_summary,
    paths.publication_tables_dir / "14_geometry_multisession_bootstrap_summa
)

display(
    geometry_multisession_summary
    .sort_values(["metric", "mean"], ascending=[True, False])
    .head(40)
)

```

	embedding	metric	n_sessions	mean	median	
32	umap	curvature_max	3	18903.736615	19429.521700	5309.7
5	isomap	curvature_max	3	1864.210022	281.032095	2885.9
14	pca	curvature_max	3	213.994987	191.321576	153.6
23	pca_full	curvature_max	3	2.417495	2.586151	0.4
30	umap	curvature_median	3	8.185327	8.253039	0.6
12	pca	curvature_median	3	1.016781	0.999325	0.0

21	pca_full	curvature_median	3	0.458323	0.448225	0.0
3	isomap	curvature_median	3	0.407156	0.366092	0.1
31	umap	curvature_p99	3	1551.896487	1647.643504	303.2
4	isomap	curvature_p99	3	58.842627	27.438089	67.2
13	pca	curvature_p99	3	30.353217	31.000895	3.1
22	pca_full	curvature_p99	3	1.491134	1.542350	0.0
2	isomap	speed_max	3	15.262419	16.518352	6.2
29	umap	speed_max	3	8.009182	6.416957	7.3
20	pca_full	speed_max	3	6.643465	6.913401	1.2
11	pca	speed_max	3	5.540066	5.884359	1.1
0	isomap	speed_median	3	2.988361	3.832981	1.5
18	pca_full	speed_median	3	2.853178	2.878449	0.1
9	pca	speed_median	3	1.104313	1.121127	0.0
27	umap	speed_median	3	0.079479	0.080179	0.0
1	isomap	speed_p99	3	10.214211	12.270895	4.1
19	pca_full	speed_p99	3	4.572585	4.561172	0.3
10	pca	speed_p99	3	3.043926	3.226026	0.4
28	umap	speed_p99	3	0.584110	0.562987	0.1
8	isomap	tangling_max	3	22433.714815	2028.882843	36515.5
35	umap	tangling_max	3	12014.475511	7908.773806	13397.8
17	pca	tangling_max	3	536.209452	446.755228	255.8
26	pca_full	tangling_max	3	4.047755	4.252986	0.4
6	isomap	tangling_median	3	10.217461	9.512631	2.9
15	pca	tangling_median	3	8.338550	8.710293	0.9
33	umap	tangling_median	3	8.270389	3.416202	8.6
24	pca_full	tangling_median	3	0.997035	0.994808	0.0
34	umap	tangling_p99	3	1990.023070	676.668744	2671.2
7	isomap	tangling_p99	3	1187.097558	477.967703	1405.7
16	pca	tangling_p99	3	182.088227	199.840421	37.5
25	pca_full	tangling_p99	3	2.827942	2.920327	0.1

```

# -----
# Notebook 14: clean multi-session geometry figure
# -----

import matplotlib.pyplot as plt

plot_metric = "curvature_median"

plot_df = geometry_multisession_summary[
    geometry_multisession_summary["metric"] == plot_metric
].copy()

if plot_df.empty:
    raise RuntimeError(f"No rows found for plot metric: {plot_metric}")

plot_df = plot_df.sort_values("mean", ascending=False).reset_index(drop=True)

fig, ax = plt.subplots(figsize=(7, 4))

yerr = np.vstack([
    plot_df["mean"] - plot_df["ci_low"],
    plot_df["ci_high"] - plot_df["mean"],
])

ax.bar(
    plot_df["embedding"],
    plot_df["mean"],
    yerr=yerr,
    capsize=4,
)

ax.set_xlabel("Embedding")
ax.set_ylabel("Mean median curvature")
ax.set_title("Trajectory curvature across V1 natural-movie sessions")
ax.tick_params(axis="x", rotation=30)

for i, row in plot_df.iterrows():
    ax.text(
        i,
        row["mean"] + 0.02 * plot_df["mean"].max(),
        f"{row['mean']:.3f}",
        ha="center",
        va="bottom",
        fontsize=9,
    )

fig.tight_layout()

save_figure(
    fig.

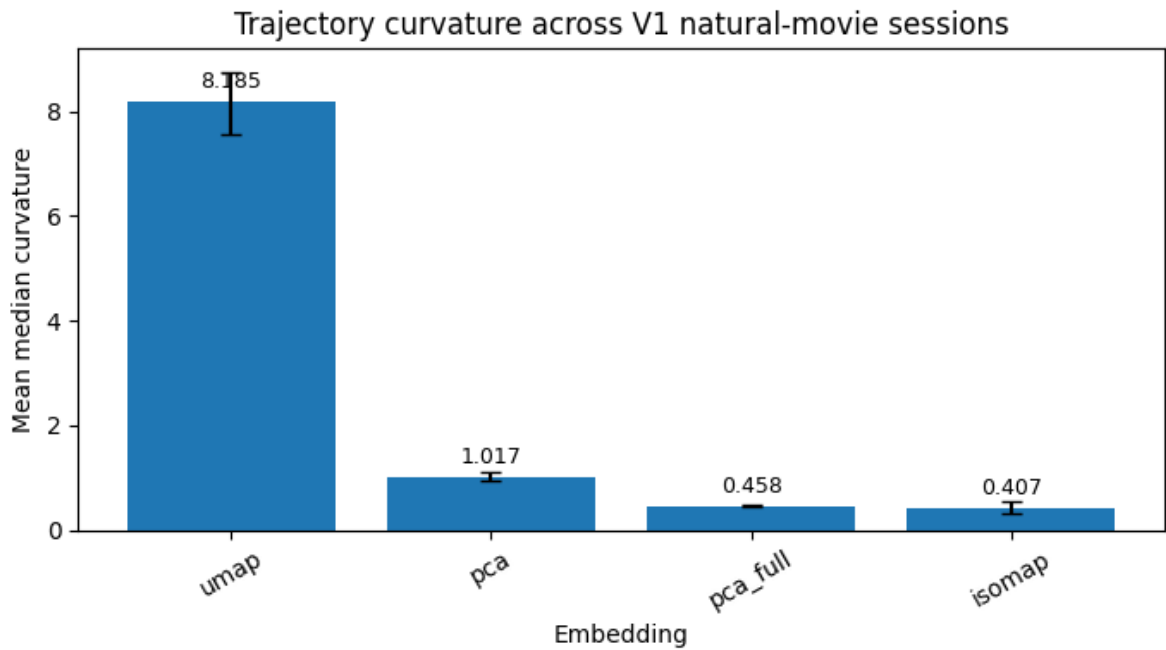
```

```

    paths.publication_figures_dir / "14_multisession_curvature_by_embedding.
)

plt.show()

```



✓ Merge session metadata and enforce publication gates

```

# -----
# Notebook 14: add session metadata and geometry claim gates
# -----

from pathlib import Path
import numpy as np
import pandas as pd

# Use the geometry table produced above.
geom_path = paths.publication_tables_dir / "14_geometry_with_session_metadat

if geom_path.exists():
    geom = pd.read_csv(geom_path)
elif "geom" in globals() and not geom.empty:
    geom = geom.copy()
else:
    raise FileNotFoundError(
        "No geometry table found. Run the geometry reconstruction cell first
    )

```

```

geom["session_id"] = geom["session_id"].astype(str)

# -----
# Add cohort/session metadata
# -----

cohort_candidates = [
    paths.publication_tables_dir / "11_publication_candidate_cohort.csv",
    paths.tables_dir / "publication_upgrade_v2" / "11_publication_candidate_
    paths.tables_dir / "publication_upgrade" / "11_publication_candidate_coh
]

cohort = pd.DataFrame()

for cohort_path in cohort_candidates:
    if cohort_path.exists() and cohort_path.stat().st_size > 0:
        try:
            cohort = pd.read_csv(cohort_path)
            print("Using cohort metadata:", cohort_path)
            break
        except Exception as exc:
            print("Could not read cohort metadata:", cohort_path, repr(exc))

if not cohort.empty and "session_id" in cohort.columns:
    cohort = cohort.copy()
    cohort["session_id"] = (
        pd.to_numeric(cohort["session_id"], errors="coerce")
        .astype("Int64")
        .astype(str)
    )

    # Avoid duplicate columns if rerunning.
    overlap = [
        c for c in cohort.columns
        if c in geom.columns and c != "session_id"
    ]

    cohort_for_merge = cohort.drop(columns=overlap, errors="ignore")

    geom_meta = geom.merge(
        cohort_for_merge,
        on="session_id",
        how="left",
    )
else:
    print("No usable cohort metadata found. Proceeding with geometry table o
    geom_meta = geom.copy()

# -----
# Add cell counts from tensor files

```

```

# -----

cell_rows = []

try:
    from v1_manifold.preprocessing import load_trial_tensor_h5

    for session_id in geom_meta["session_id"].astype(str).unique():
        match = asset_index[
            asset_index["session_id"].astype(str) == str(session_id)
        ]

        if match.empty or "tensor_file" not in match.columns:
            continue

        tensor_file = match["tensor_file"].iloc[0]

        if pd.isna(tensor_file):
            continue

        tensor_path = Path(tensor_file)

        if tensor_path.exists():
            _, R = load_trial_tensor_h5(tensor_path)
            cell_rows.append({
                "session_id": str(session_id),
                "n_cells": int(R.shape[1]),
            })

except Exception as exc:
    print("Could not add cell counts:", repr(exc))

if cell_rows:
    cell_counts = pd.DataFrame(cell_rows).drop_duplicates("session_id")

    if "n_cells" in geom_meta.columns:
        geom_meta = geom_meta.drop(columns=["n_cells"])

    geom_meta = geom_meta.merge(
        cell_counts,
        on="session_id",
        how="left",
    )

save_table(
    geom_meta,
    paths.publication_tables_dir / "14_geometry_with_session_metadata.csv",
)

display(geom_meta.head(20))

```

```

# -----
# Claim gates
# -----

n_sessions = (
    geom_meta["session_id"].nunique()
    if not geom_meta.empty
    else 0
)

n_layers = (
    geom_meta["putative_layer"].dropna().nunique()
    if "putative_layer" in geom_meta.columns
    else 0
)

n_cre = (
    geom_meta["cre_line"].dropna().nunique()
    if "cre_line" in geom_meta.columns
    else 0
)

n_embeddings = (
    geom_meta["embedding"].dropna().nunique()
    if "embedding" in geom_meta.columns
    else 0
)

n_sessions_with_cells = (
    geom_meta.loc[geom_meta["n_cells"].notna(), "session_id"].nunique()
    if "n_cells" in geom_meta.columns
    else 0
)

gate_rows = [
    {
        "gate": "multi_session_geometry",
        "status": "pass" if n_sessions >= 3 else "fail",
        "value": n_sessions,
        "criterion": ">=3 fully embedded sessions",
        "interpretation": "Allows cautious multi-session geometry summaries."
    },
    {
        "gate": "common_embedding_geometry",
        "status": "pass" if n_embeddings >= 4 else "fail",
        "value": n_embeddings,
        "criterion": ">=4 common embeddings reconstructed",
        "interpretation": "Allows comparison across PCA, PCA-full, UMAP, and

```

```

        "gate": "cell_count_available",
        "status": "pass" if n_sessions_with_cells >= 3 else "fail",
        "value": n_sessions_with_cells,
        "criterion": "cell counts available for >=3 sessions",
        "interpretation": "Allows cell-count-aware reporting and controls.",
    },
    {
        "gate": "multi_layer_geometry",
        "status": "pass" if n_layers >= 3 else "fail",
        "value": n_layers,
        "criterion": ">=3 represented layers",
        "interpretation": "Do not claim layer-specific geometry unless this
    },
    {
        "gate": "multi_cre_geometry",
        "status": "pass" if n_cre >= 2 else "fail",
        "value": n_cre,
        "criterion": ">=2 Cre lines",
        "interpretation": "Do not claim Cre-line-specific geometry unless th
    },
]

geometry_gates = pd.DataFrame(gate_rows)

save_table(
    geometry_gates,
    paths.publication_tables_dir / "14_geometry_claim_gates.csv",
)

display(geometry_gates)

```

Using cohort metadata: C:\Users\Peter\Documents\projects\NeuroAI\latent-manif

	session_id	embedding	n_frames	embedding_dim	speed_median	speed_p99
0	500855614	pca	900	3	1.005158	2.503851
1	500855614	pca_full	900	20	2.677863	4.246316
2	500855614	umap	900	3	0.082220	0.752321
3	500855614	isomap	900	3	1.200051	5.479009
4	500964514	pca	900	3	1.121127	3.401901

5	500964514	pca_full	900	20	3.003222	4.910268
6	500964514	umap	900	3	0.080179	0.437021
7	500964514	isomap	900	3	3.832981	12.892728
8	501271265	pca	900	3	1.186652	3.226026
9	501271265	pca_full	900	20	2.878449	4.561172
10	501271265	umap	900	3	0.076037	0.562987
11	501271265	isomap	900	3	3.932050	12.270895

12 rows × 28 columns

		gate	status	value	criterion	interpretation
0	multi_session_geometry		pass	3	>=3 fully embedded sessions	Allows cautious multi-session geometry summaries.
1	common_embedding_geometry		pass	4	>=4 common embeddings reconstructed	Allows comparison across PCA, PCA-full, UMAP, ...
2	cell count available		pass	3	cell counts available for	Allows cell-count-aware reporting

✓ Bootstrap uncertainty by embedding, layer, and Cre line

```
# -----
# Notebook 14: valid geometry bootstrap CIs by embedding only
# -----

from v1_manifold_publication.stats import bootstrap_ci
import numpy as np
import pandas as pd
```



```

# Use only columns that summarize geometry distributions.
value_cols = [
    c for c in geom_meta.columns
    if c.endswith("_median") or c.endswith("_p99") or c.endswith("_max")
]

summary_rows = []

if geom_meta.empty:
    raise RuntimeError("geom_meta is empty. Run the geometry metadata/claim-

for col in value_cols:
    for embedding, sub in geom_meta.groupby("embedding", dropna=False):
        values = pd.to_numeric(sub[col], errors="coerce").dropna().to_numpy()

        if len(values) == 0:
            continue

        ci = bootstrap_ci(
            values,
            n_bootstrap=int(cfg["models"].get("n_bootstrap", 1000)),
            rng=cfg["project"]["random_seed"],
        )

        summary_rows.append({
            "embedding": embedding,
            "metric": col,
            "n_sessions": int(sub["session_id"].nunique()),
            "mean": float(np.mean(values)),
            "median": float(np.median(values)),
            "std": float(np.std(values, ddof=1)) if len(values) > 1 else 0.0
            "ci_low": ci["ci_low"],
            "ci_high": ci["ci_high"],
            "claim_scope": "multi_session_embedding_geometry",
            "claim_status": "supported" if sub["session_id"].nunique() >= 3
        })

geom_ci = pd.DataFrame(summary_rows)

save_table(
    geom_ci,
    paths.publication_tables_dir / "14_geometry_bootstrap_ci_by_embedding.csv"
)

display(
    geom_ci
    .sort_values(["metric", "mean"], ascending=[True, False])
    .head(40)
)

```

	embedding	metric	n_sessions	mean	median	
23	umap	curvature_max	3	18903.736615	19429.521700	5309.7
20	isomap	curvature_max	3	1864.210022	281.032095	2885.9
21	pca	curvature_max	3	213.994987	191.321576	153.6
22	pca_full	curvature_max	3	2.417495	2.586151	0.4
15	umap	curvature_median	3	8.185327	8.253039	0.6
13	pca	curvature_median	3	1.016781	0.999325	0.0
14	pca_full	curvature_median	3	0.458323	0.448225	0.0
12	isomap	curvature_median	3	0.407156	0.366092	0.1
19	umap	curvature_p99	3	1551.896487	1647.643504	303.2
16	isomap	curvature_p99	3	58.842627	27.438089	67.2
17	pca	curvature_p99	3	30.353217	31.000895	3.1
18	pca_full	curvature_p99	3	1.491134	1.542350	0.0
8	isomap	speed_max	3	15.262419	16.518352	6.2
11	umap	speed_max	3	8.009182	6.416957	7.3
10	pca_full	speed_max	3	6.643465	6.913401	1.2
9	pca	speed_max	3	5.540066	5.884359	1.1
0	isomap	speed_median	3	2.988361	3.832981	1.5
2	pca_full	speed_median	3	2.853178	2.878449	0.1
1	pca	speed_median	3	1.104313	1.121127	0.0
3	umap	speed_median	3	0.079479	0.080179	0.0
4	isomap	speed_p99	3	10.214211	12.270895	4.1
6	pca_full	speed_p99	3	4.572585	4.561172	0.3
5	pca	speed_p99	3	3.043926	3.226026	0.4
7	umap	speed_p99	3	0.584110	0.562987	0.1
32	isomap	tangling_max	3	22433.714815	2028.882843	36515.5
35	umap	tangling_max	3	12014.475511	7908.773806	13397.8
33	pca	tangling_max	3	536.209452	446.755228	255.8
34	pca_full	tangling_max	3	4.047755	4.252986	0.4
24	isomap	tangling_median	3	10.217461	9.512631	2.9
25	pca	tangling median	3	8.338550	8.710293	0.9

27	umap	tangling_median	3	8.270389	3.416202	8.6
26	pca_full	tangling_median	3	0.997035	0.994808	0.0
31	umap	tangling_p99	3	1990.023070	676.668744	2671.2
28	isomap	tangling_p99	3	1187.097558	477.967703	1405.7
29	pca	tangling_p99	3	182.088227	199.840421	37.5
30	pca_full	tangling_p99	3	2.827942	2.920327	0.1

```
# -----
# Notebook 14: blocked biological subgroup claims
# -----

blocked_rows = []

n_layers = (
    geom_meta["putative_layer"].dropna().nunique()
    if "putative_layer" in geom_meta.columns
    else 0
)

n_cre = (
    geom_meta["cre_line"].dropna().nunique()
    if "cre_line" in geom_meta.columns
    else 0
)

blocked_rows.append({
    "claim": "layer_specific_geometry",
    "status": "blocked",
    "reason": f"Only {n_layers} putative layer(s) represented.",
    "required_for_claim": "At least 3 represented layers with enough session
})

blocked_rows.append({
    "claim": "cre_line_specific_geometry",
    "status": "blocked",
    "reason": f"Only {n_cre} Cre line(s) represented.",
    "required_for_claim": "At least 2 Cre lines with enough sessions per Cre
})

blocked_geometry_claims = pd.DataFrame(blocked_rows)

save_table(
    blocked_geometry_claims,
    paths.publication_tables_dir / "14_blocked_layer_cre_geometry_claims.csv
)
```

```
display(blocked_geometry_claims)
```

	claim	status	reason	required_for_claim
0	layer_specific_geometry	blocked	Only 1 putative layer(s) represented.	At least 3 represented layers with enough sess...

✓ Cell-count-matched PCA geometry across sessions

```
# -----
# Notebook 14: manual cell-count-matched PCA geometry control
# -----
# This replaces the failed cell_count_matched_metric_table(...) call.
# It samples the same number of cells from each session and computes
# PCA participation ratio for each subsample.
# -----

import numpy as np
import pandas as pd

from v1_manifold_publication.stats import bootstrap_ci

# -----
# Check that population_by_session exists from the previous successful load
# -----

if "population_by_session" not in globals() or not population_by_session:
    raise RuntimeError(
        "population_by_session is missing. Re-run the tensor-loading part of
    )

# -----
# Settings
# -----

available_cell_counts = {
    str(session_id): int(R.shape[1])
    for session_id, R in population_by_session.items()
}

min_available = min(available_cell_counts.values())

requested = cfg.get("models", {}).get("matched_cell_count", None)

matched_cell_count = int(
    min(requested or min_available, min_available)
)
```

```

/

n_subsamples = int(
    cfg.get("models", {}).get("n_cell_subsamples", 100)
)

rng = np.random.default_rng(cfg["project"]["random_seed"])

print("Available cell counts:", available_cell_counts)
print("Matched cell count:", matched_cell_count)
print("Number of subsamples:", n_subsamples)

# -----
# Metric
# -----

def pca_geometry_metric(X):
    """
    Compute PCA participation ratio from a frame-by-cell matrix.

    X shape: [n_frames, n_cells]
    """
    X = np.asarray(X, dtype=float)

    if X.ndim != 2:
        raise ValueError(f"Expected 2D matrix, got shape {X.shape}")

    X = X - X.mean(axis=0, keepdims=True)

    s = np.linalg.svd(X, full_matrices=False, compute_uv=False)

    lambdas = s**2 / max(X.shape[0] - 1, 1)

    participation_ratio = (
        (lambdas.sum() ** 2)
        / (np.sum(lambdas**2) + 1e-12)
    )

    return {
        "pca_participation_ratio": float(participation_ratio),
        "n_frames": int(X.shape[0]),
    }

# -----
# Manual matched-cell subsampling
# -----

matched_rows = []

for session_id, R in population_by_session.items():
    session_id = str(session_id)

```

```

R = np.asarray(R, dtype=float)

n_available_cells = R.shape[1]

if n_available_cells < matched_cell_count:
    print(
        f"Skipping session {session_id}: only {n_available_cells} cells,
        f"need {matched_cell_count}."
    )
    continue

print(f"Sampling session {session_id}: R={R.shape}")

for subsample in range(n_subsamples):
    cell_indices = rng.choice(
        n_available_cells,
        size=matched_cell_count,
        replace=False,
    )

    X_sub = R[:, cell_indices]

    metrics = pca_geometry_metric(X_sub)

    matched_rows.append({
        "session_id": session_id,
        "subsample": subsample,
        "matched_cell_count": matched_cell_count,
        "available_cell_count": n_available_cells,
        "cell_indices": " ".join(map(str, cell_indices.tolist())),
        **metrics,
    })

matched_geometry = pd.DataFrame(matched_rows)

# -----
# Add metadata
# -----

if not matched_geometry.empty and "geom_meta" in globals() and not geom_meta:
    metadata_cols = [
        c for c in [
            "session_id",
            "putative_layer",
            "cre_line",
            "imaging_depth",
            "session_type",
            "donor_name",
        ]
        if c in geom_meta.columns
    ]

```

```

J

if "session_id" in metadata_cols:
    meta_unique = (
        geom_meta[metadata_cols]
        .copy()
        .drop_duplicates("session_id")
    )

    meta_unique["session_id"] = meta_unique["session_id"].astype(str)
    matched_geometry["session_id"] = matched_geometry["session_id"].astype(str)

    matched_geometry = matched_geometry.merge(
        meta_unique,
        on="session_id",
        how="left",
    )

matched_geometry["n_subsamples"] = n_subsamples
matched_geometry["claim_scope"] = "cell_count_matched_session_control"

save_table(
    matched_geometry,
    paths.publication_tables_dir / "14_cell_count_matched_pca_geometry.csv",
)

display(matched_geometry.head(20))

# -----
# Session-level bootstrap summaries
# -----

session_summary_rows = []

for session_id, sub in matched_geometry.groupby("session_id", dropna=False):
    values = (
        pd.to_numeric(sub["pca_participation_ratio"], errors="coerce")
        .dropna()
        .to_numpy()
    )

    if len(values) == 0:
        continue

    ci = bootstrap_ci(
        values,
        n_bootstrap=int(cfg["models"].get("n_bootstrap", 1000)),
        rng=cfg["project"]["random_seed"],
    )

    session_summary_rows.append({

```

```

        "session_id": str(session_id),
        "n_subsamples": int(len(values)),
        "matched_cell_count": matched_cell_count,
        "mean_pca_participation_ratio": float(np.mean(values)),
        "median_pca_participation_ratio": float(np.median(values)),
        "std_pca_participation_ratio": float(np.std(values, ddof=1)) if len(
            "ci_low": ci["ci_low"],
            "ci_high": ci["ci_high"],
            "claim_scope": "within_session_cell_count_matched_stability",
    })

matched_session_summary = pd.DataFrame(session_summary_rows)

save_table(
    matched_session_summary,
    paths.publication_tables_dir / "14_cell_count_matched_pca_geometry_sessi
)

display(matched_session_summary)

# -----
# Across-session summary
# -----

session_means = (
    matched_session_summary["mean_pca_participation_ratio"]
    .dropna()
    .to_numpy()
)

if len(session_means):
    across_ci = bootstrap_ci(
        session_means,
        n_bootstrap=int(cfg["models"].get("n_bootstrap", 1000)),
        rng=cfg["project"]["random_seed"],
    )

    matched_across_session_summary = pd.DataFrame([
        "n_sessions": int(len(session_means)),
        "matched_cell_count": matched_cell_count,
        "mean_session_pca_participation_ratio": float(np.mean(session_means))
        "median_session_pca_participation_ratio": float(np.median(session_means))
        "std_session_pca_participation_ratio": float(np.std(session_means, ddof=1))
        "ci_low": across_ci["ci_low"],
        "ci_high": across_ci["ci_high"],
        "claim_scope": "cell_count_matched_across_session_summary",
    ])
else:
    matched_across_session_summary = pd.DataFrame()

save_table(

```



```
save_table(\n    matched_across_session_summary,\n    paths.publication_tables_dir / "14_cell_count_matched_pca_geometry_acros\n)\n\ndisplay(matched_across_session_summary)\n\nprint("Saved:")\nprint(paths.publication_tables_dir / "14_cell_count_matched_pca_geometry.csv\nprint(paths.publication_tables_dir / "14_cell_count_matched_pca_geometry_ses\nprint(paths.publication_tables_dir / "14_cell_count_matched_pca_geometry_acr
```

Available cell counts: {'500855614': 163, '500964514': 214, '501271265': 215}\nMatched cell count: 50\nNumber of subsamples: 100\nSampling session 500855614: R=(900, 163)\nSampling session 500964514: R=(900, 214)\nSampling session 501271265: R=(900, 215)

	session_id	subsample	matched_cell_count	available_cell_count	cell_ind
0	500855614	0	50	163	56 66 5 162 158 109 42 7 161 11
1	500855614	1	50	163	147 58 95 90 64 4 157 59 79 11
2	500855614	2	50	163	101 28 59 25 119 15 154 83 9 1
3	500855614	3	50	163	14 161 45 48 87 128 43 7 124 1
4	500855614	4	50	163	127 2 17 120 59 6 108 62 3 87 1
5	500855614	5	50	163	135 37 108 56 112 70 9 50 88 49
6	500855614	6	50	163	36 16 7 110 148 7 94 51 103 138 70 145

7	500855614	7	50	163	104 29 102 75 9 61 68 72
8	500855614	8	50	163	130 15 138 99 12 37 69 13 46 14
9	500855614	9	50	163	150 5 74 12 141 64 45 151 39 82
10	500855614	10	50	163	51 92 70 140 63 96 110 49 14 14
11	500855614	11	50	163	99 148 159 10 101 39 9 126 27
12	500855614	12	50	163	116 35 104 20 94 72 138 6 10 14
13	500855614	13	50	163	119 56 16 96 70 11 71 57 127 40
14	500855614	14	50	163	125 42 7 123 63 20 90 36 8 78 1
15	500855614	15	50	163	0 126 4 28 104 14 1 21 145 69 4
16	500855614	16	50	163	2 17 11 149 15 156 96 6 9 8 135
17	500855614	17	50	163	142 12 34 33 2 94 11 8 128 70
18	500855614	18	50	163	47 147 6 44 16 14 160 135 12 2

				42 2
				29 67 13
19	500855614	19	50	163 134 145 7
				157 45 2
				41 1
	session_id	n_subsamples	matched_cell_count	mean_pca_participation_ratio
0	500855614	100	50	9.175682
1	500964514	100	50	8.491070
2	501271265	100	50	9.006799
	n_sessions	matched_cell_count	mean_session_pca_participation_ratio	medi
0	3	50		8.891184

Saved:
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\r
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\r
C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-movies\r

Layer/Cre-line comparison table and figures

```

# -----
# Notebook 14: geometry statistics and publication figures
# -----
# Layer/Cre statistics are blocked because only one layer and one Cre line
# are represented. We still generate valid embedding-level geometry figures.
# -----

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from v1_manifold_publication.plotting import bar_with_points, savefig

# -----
# Block layer/Cre inferential statistics unless gates pass
# -----

geometry_gates_path = paths.publication_tables_dir / "14_geometry_claim_gate

if geometry_gates_path.exists():
    geometry_gates = pd.read_csv(geometry_gates_path)
else:
    geometry_gates = pd.DataFrame()

```

```

def gate_passed(gate_name):
    if geometry_gates.empty:
        return False

    rows = geometry_gates[geometry_gates["gate"] == gate_name]

    if rows.empty:
        return False

    return str(rows["status"].iloc[0]).lower() == "pass"

layer_gate = gate_passed("multi_layer_geometry")
cre_gate = gate_passed("multi_cre_geometry")

stats_rows = []

if geom_meta.empty:
    print("geom_meta is empty. Run the metadata/claim-gate cell first.")

else:
    geometry_metrics = [
        c for c in geom_meta.columns
        if c.endswith("_median")
    ]

    for metric in geometry_metrics:
        for factor, gate_ok in [
            ("putative_layer", layer_gate),
            ("cre_line", cre_gate),
        ]:
            if factor not in geom_meta.columns:
                stats_rows.append({
                    "factor": factor,
                    "metric": metric,
                    "status": "blocked",
                    "reason": f"{factor} column not available.",
                    "n_groups": 0,
                    "n_sessions": int(geom_meta["session_id"].nunique()),
                })
                continue

            n_groups = int(geom_meta[factor].dropna().nunique())
            n_sessions = int(geom_meta["session_id"].nunique())

            if not gate_ok:
                stats_rows.append({
                    "factor": factor,
                    "metric": metric,
                    "status": "blocked",
                    ..
                })

```

```

        "reason": f"{factor} comparison blocked by claim gates."
        "n_groups": n_groups,
        "n_sessions": n_sessions,
        "count": np.nan,
        "mean": np.nan,
        "median": np.nan,
        "std": np.nan,
    })
    continue

groups = geom_meta.dropna(subset=[factor, metric]).groupby(factor)

if groups.ngroups < 2:
    stats_rows.append({
        "factor": factor,
        "metric": metric,
        "status": "blocked",
        "reason": "Fewer than two groups available.",
        "n_groups": int(groups.ngroups),
        "n_sessions": n_sessions,
        "count": np.nan,
        "mean": np.nan,
        "median": np.nan,
        "std": np.nan,
    })
    continue

summary = (
    groups[metric]
    .agg(["count", "mean", "median", "std"])
    .reset_index()
)

for _, row in summary.iterrows():
    stats_rows.append({
        "factor": factor,
        "factor_level": row[factor],
        "metric": metric,
        "status": "computed",
        "reason": "Claim gate passed.",
        "n_groups": int(groups.ngroups),
        "n_sessions": n_sessions,
        "count": int(row["count"]),
        "mean": float(row["mean"]),
        "median": float(row["median"]),
        "std": float(row["std"]) if pd.notna(row["std"]) else np

layer_or_cre_stats = pd.DataFrame(stats_rows)

save table(

```

```

        layer_or_cre_stats,
        paths.publication_tables_dir / "14_layer_or_cre_geometry_statistics.csv"
    )

display(layer_or_cre_stats.head(30))

# -----
# Valid embedding-level figures
# -----

if not geom_meta.empty and "curvature_median" in geom_meta.columns:
    fig = bar_with_points(
        geom_meta,
        x="embedding",
        y="curvature_median",
        title="Median trajectory curvature across V1 natural-movie sessions"
    )

    savefig(
        fig,
        paths.publication_figures_dir / "14_median_curvature_by_embedding.png"
    )

    plt.show()

if not geom_meta.empty and "tangling_median" in geom_meta.columns:
    fig = bar_with_points(
        geom_meta,
        x="embedding",
        y="tangling_median",
        title="Median trajectory tangling across V1 natural-movie sessions",
    )

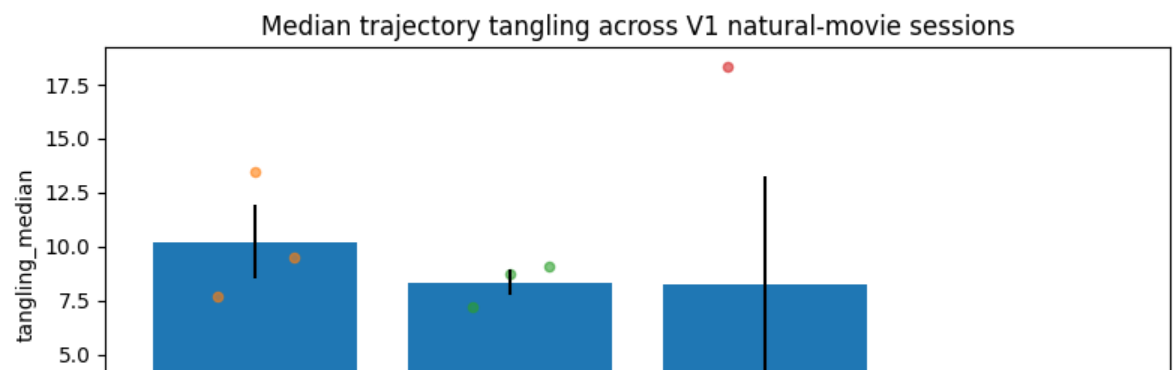
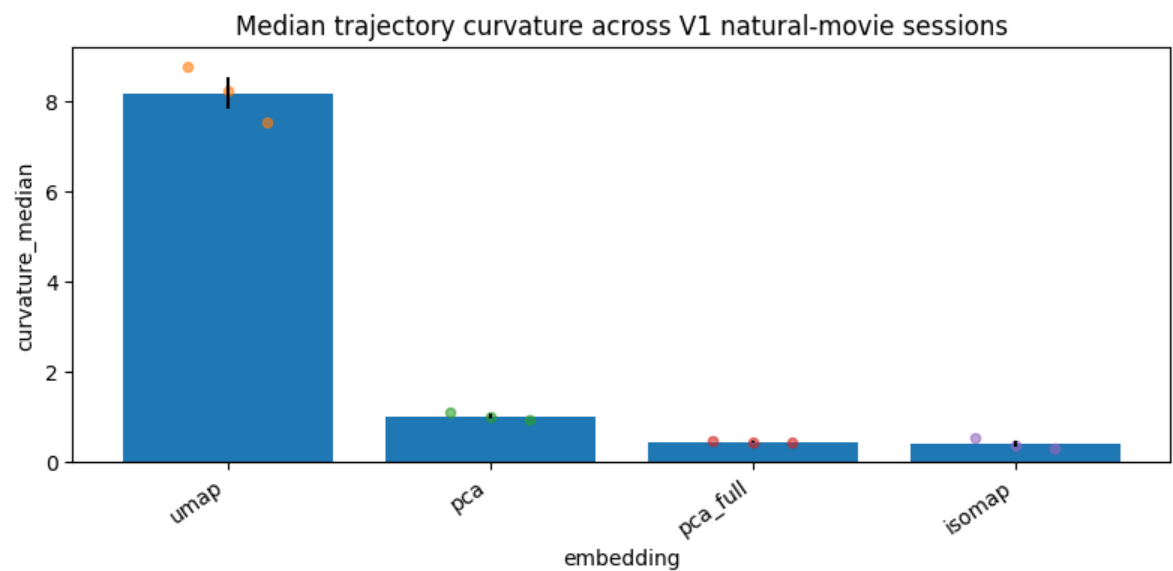
    savefig(
        fig,
        paths.publication_figures_dir / "14_median_tangling_by_embedding.png"
    )

    plt.show()

```

	factor	metric	status	reason	n_groups	n_sessions	c
0	putative_layer	speed_median	blocked	putative_layer comparison blocked by claim gates.	1	3	
1	cre_line	speed_median	blocked	cre_line comparison blocked by claim gates.	1	3	

2	putative_layer	curvature_median	blocked	putative_layer comparison blocked by claim gates.	1	3
3	cre_line	curvature_median	blocked	cre_line comparison blocked by claim gates.	1	3
4	putative_layer	tangling_median	blocked	putative_layer comparison blocked by claim gates.	1	3
5	cre_line	tangling_median	blocked	cre_line comparison blocked by claim gates.	1	3



```
# -----
# Notebook 14 completion checkpoint
# -----

expected_outputs = [
    "publication_reconstructed_embedding_geometry.csv",
    "14_geometry_with_session_metadata.csv"
```

```

14_geometry_with_session_metadata.csv",
"14_geometry_reconstruction_status.csv",
"14_geometry_multisession_bootstrap_summary.csv",
"14_geometry_bootstrap_ci_by_embedding.csv",
"14_geometry_claim_gates.csv",
"14_blocked_layer_cre_geometry_claims.csv",
"14_layer_or_cre_geometry_statistics.csv",
"14_cell_count_matched_pca_geometry.csv",
"14_cell_count_matched_pca_geometry_session_summary.csv",
"14_cell_count_matched_pca_geometry_across_session_summary.csv",
]

rows = []

for name in expected_outputs:
    path = paths.publication_tables_dir / name
    rows.append({
        "file": name,
        "exists": path.exists(),
        "size_kb": round(path.stat().st_size / 1024, 2) if path.exists() else 0
    })

notebook14_checkpoint = pd.DataFrame(rows)

save_table(
    notebook14_checkpoint,
    paths.publication_tables_dir / "14_notebook_completion_checkpoint.csv",
)

display(notebook14_checkpoint)

```

	file	exists	size_kb
0	publication_reconstructed_embedding_geometry.csv	True	2.39
1	14_geometry_with_session_metadata.csv	True	4.72
2	14_geometry_reconstruction_status.csv	True	0.44
3	14_geometry_multisession_bootstrap_summary.csv	True	4.10
4	14_geometry_bootstrap_ci_by_embedding.csv	True	5.64
5	14_geometry_claim_gates.csv	True	0.59
6	14_blocked_layer_cre_geometry_claims.csv	True	0.29
7	14_layer_or_cre_geometry_statistics.csv	True	0.60
8	14_cell_count_matched_pca_geometry.csv	True	87.75
9	14_cell_count_matched_pca_geometry_session_sum...	True	0.60
10	14_cell_count_matched_pca_geometry_across_sess...	True	0.30

